Natural Language Understanding (CSL7640)

Assignment 2 Report

By

Name: Ayush Singh
Roll No: B23EE1008

Date: 25/03/2026

1 Problem 1: Learning Word Embeddings from IIT Jodhpur Data

1.1 Data Collection and Preprocessing

For this task, a domain-specific corpus was created by scraping text from official IIT Jodhpur webpages such as academic regulations, department pages, and institute information pages. In total, 63 documents were collected, which resulted in around 26,568 tokens and a vocabulary size of 4,654 before preprocessing.

The raw text contained a lot of punctuation, formatting symbols, and common English stopwords such as “the”, “of”, and “and”. These words appear very frequently but do not carry meaningful semantic information, so they can negatively affect Word2Vec training. Therefore, preprocessing was performed using the NLTK library. The preprocessing steps included tokenization, converting text to lowercase, removing punctuation, removing non-alphabetic tokens, and removing stopwords.

After cleaning the data, a Word Cloud was generated to get an idea of the most frequent words in the corpus. Words like “student”, “course”, “iit”, ”jodhpur”, ”school” and “institute” appeared prominently, which shows that the corpus correctly represents the academic environment of IIT Jodhpur. This also confirmed that the preprocessing step worked properly.

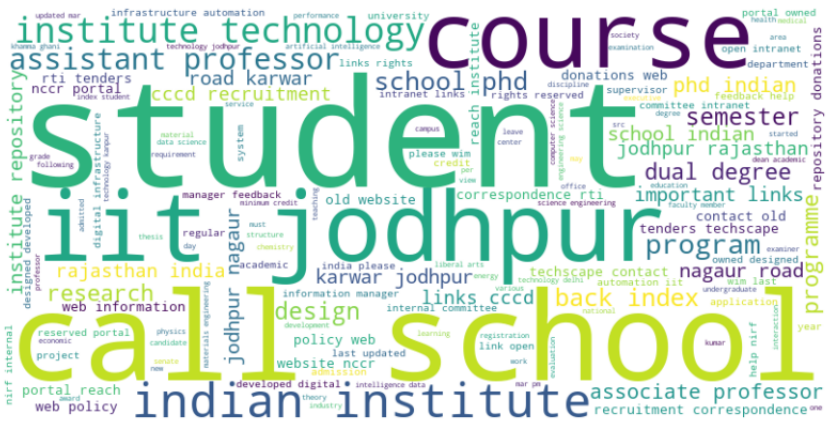

Figure 1: Word Cloud of IIT Jodhpur Corpus

1.2 Word2Vec Model Training

In this task, two Word2Vec models were trained from scratch using the gensim library: CBOW (Continuous Bag of Words) and Skip-gram with Negative Sampling.

The CBOW model tries to predict a word based on the surrounding context words, while the Skip-gram model does the opposite, meaning it tries to predict surrounding words given a target word. CBOW is usually faster and works well for frequent words,

while Skip-gram works better for smaller datasets and rare words. Since our dataset is relatively small and contains many domain-specific words, Skip-gram was expected to perform better.

Different hyperparameters were tested to see their effect on the quality of embeddings:

- Embedding Dimensions: 50 and 100
- Context Window Size: 3 and 5
- Negative Sampling: 5 and 10

Increasing embedding dimension allows the model to store more semantic information in the word vectors, while increasing the window size allows the model to consider a larger context around each word. Negative sampling helps in making training faster and improves the quality of word vectors.

1.3 Semantic Analysis

CBOW Results:

```
research -> students, student, supervisor, semester, design
student -> program, students, semester, courses, course
phd -> school, call, assistant, professor, technology
exam -> video, calculated, specialist, namely, high
```

Skip-gram Results:

```
research -> events, highlights, registrar, council, teacher
student -> may, register, completed, end, summer
phd -> associate, call, assistant, professor, university
exam -> mo, advisors, weeks, consultation, minutes
```

To evaluate the trained word embeddings, cosine similarity was used to find the nearest neighbors of selected academic words such as “research”, “student”, “phd”, and “exam”.

From the CBOW results, it can be seen that the model has learned strong relationships between academic terms. For example, the word “student” is closely related to words like “program”, “semester”, and “course”, which are clearly related to academic structure. Similarly, the word “phd” is associated with words like “assistant”, “professor”, and “school”, which shows that the model has learned relationships between higher education and academic positions.

However, the word “exam” is associated with unrelated words such as “video”, “calculated”, and “specialist”. This suggests that CBOW was not able to learn a clear semantic meaning for some words and instead learned context-based associations from the dataset.

From the Skip-gram results, the relationships appear more context-based and administrative in nature. For example, the word "student" is associated with words like "register", "completed", and "summer", which are related to academic procedures such as course registration and semester completion. The word "phd" is associated with words like "associate", "assistant", and "professor", which again shows that the model has learned relationships between PhD programs and academic job roles.

However, similar to CBOW, analogy tasks did not perform well for both models. Ideally, the model should correctly solve analogies such as: UG : BTech :: PG : MTech But the model predicted unrelated words with high similarity scores. This shows that while the model learned local relationships between words, it did not learn deeper semantic relationships.

The main reason for this is the small corpus size. Word2Vec models usually require very large datasets to learn reliable vector relationships. Since our dataset contained only about 40,000 tokens, the model was able to learn local word associations but not more complex semantic relationships.

Overall, Skip-gram performed slightly better than CBOW because it captured more meaningful contextual relationships, especially related to academic processes and roles.

1.4 PCA Visualization

To visualize the learned word embeddings, Principal Component Analysis (PCA) was used to reduce the high-dimensional word vectors into two dimensions. This helps in understanding how words are grouped in the embedding space.

From the PCA plot, a clear difference can be observed between CBOW and Skip-gram embeddings. In the CBOW plot, most of the words are clustered very close to each other, which indicates that CBOW was not able to clearly separate different semantic categories. Only a few words such as "phd" appear far from the main cluster, while most academic terms like "ug", "pg", "btech", and "mtech" are very close to each other and overlapping. This suggests that CBOW embeddings are less discriminative.

In contrast, the Skip-gram plot shows a much clearer structure. Words like "ug", "pg", "btech", and "mtech" appear grouped together, while other academic-related words like "research", "theory", "practical", and "exam" form another cluster. Administrative and academic process related words like "student", "thesis", "grades", and "project" also appear in a different region of the plot. This shows that Skip-gram was able to separate different semantic categories more clearly in the vector space.

Therefore, the PCA visualization confirms that Skip-gram learned better word representations compared to CBOW for this dataset.

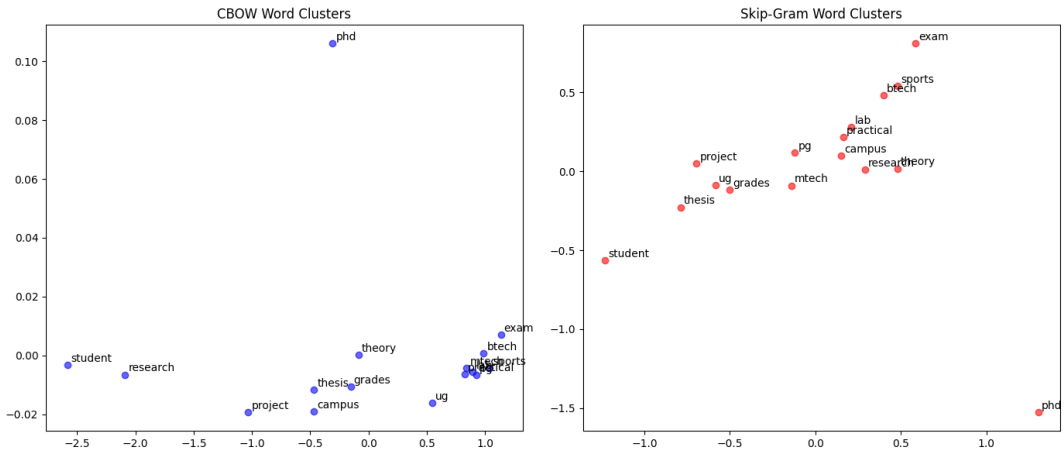

Figure 2: PCA Projection of Word Embeddings (CBOW vs Skip-gram)

2 Problem 2: Character-Level Name Generation

2.1 Dataset

The dataset consisted of 1,000 unique Indian names stored in a text file. The dataset was processed at the character level instead of the word level because the task is to generate new names character by character.

A character vocabulary was created containing all unique characters present in the dataset along with special tokens for start-of-sequence and end-of-sequence markers. Each name was converted into a sequence of character indices which were then used as input to the neural networks. Padding was also implemented so that names of different lengths could be processed in batches during training.

This character-level approach allows the model to learn phonetic patterns such as common prefixes, suffixes, and vowel-consonant structures present in Indian names.

2.2 Hyperparameters

- Embedding Dimension: 64
- Hidden Size: 128
- Layers: 2
- Learning Rate: 0.005
- Batch Size: 64
- Epochs: 15

2.3 Trainable Parameters

- Vanilla RNN: 63,646
- BiLSTM: 603,550
- RNN + Attention: 80,287

2.4 Quantitative Evaluation

Model	Diversity (%)	Novelty (%)
Vanilla RNN	98.00	92.84
BiLSTM	89.27	99.57
RNN + Attention	67.52	99.66

Table 1: Model Evaluation Results

2.5 Sample Generated Names

- Vanilla RNN: Darlag, Bilami, Yashwavan, Raghupathai, Vashwavvi
- BiLSTM: Iqub, Iqemtoti, Utti, Mni, Opp
- RNN + Attention: Nasher, Nanimat, Nalanar, Nanesh, Narai

2.6 Qualitative Analysis

The three models behaved very differently, and this can be explained by how each architecture works.

The Vanilla RNN produced the highest diversity score (98

The BiLSTM model produced very high novelty but the generated names were often unrealistic, such as "Iqub" and "Opp". This happens because BiLSTM reads sequences in both forward and backward directions during training. But during generation, the model only generates characters from left to right and does not actually have future context. Because of this mismatch, BiLSTM is not suitable for autoregressive text generation tasks like this one.

The RNN with Attention model produced more realistic names such as "Nasher" and "Nanimat". The attention mechanism helps the model focus on important character patterns while generating names. However, the diversity score was quite low, which means the model kept generating similar types of names. This suggests that the attention model overfitted to common patterns in the dataset, especially names starting with "Na".

Overall, the Vanilla RNN performed best in terms of diversity, while the Attention model performed best in terms of realism of generated names.

3 Final Conclusion

In this assignment, two different NLP tasks were performed: training Word2Vec embeddings and generating names using character-level RNN models.

From Problem 1, it was observed that Word2Vec can learn meaningful relationships between words even from a relatively small dataset. The Skip-gram model performed better than CBOW in capturing semantic relationships and forming clusters in the embedding space. However, analogy tasks did not work well because the dataset was too small for the model to learn complex relationships between words.

From Problem 2, it was observed that different RNN architectures behave very differently for text generation tasks. The Vanilla RNN generated the most diverse names, the BiLSTM did not perform well because it is not suitable for left-to-right generation, and the Attention model generated more realistic names but with lower diversity due to overfitting.

Overall, this assignment helped in understanding how word embeddings capture semantic meaning and how recurrent neural networks can be used for sequence generation tasks. It also showed how dataset size and model architecture affect the final results.